

ArchFlow E1 Block 1 Execution Plan

Prepared 2026-06-25 | Public-safe project report | Scope: E1, E1.1, E1.2, E1.3

Executive Summary

E1 is the active Block 1 workstream: build a clean knowledge base on ArchFlow itself, prove one dialogue-to-PRD loop, and write the approved PRD plus agent history into the KB. The correct operating rule is sequential installation and validation, not bulk setup. Each layer must earn the next layer by producing a concrete output and passing a small check.

The practical order is: Git safety -> env/config -> Ollama -> LangSmith -> LangGraph -> Pydantic/YAML validation -> LlamaIndex -> CrewAI -> observability/provenance papers.

Standup Status

Area	Status	Meaning
Done / verified	Git safety, ignored local env/runtime files, workflow YAML parse, skill validation, Notion page updates, subtask records	The planning and routing layer is now concrete and visible.
In progress	E1.1 setup boundary	E1.1 is now marked In Progress in Notion; E1.1.1 is in Review and E1.1.2 is In Progress.
Next	Ollama, LangSmith, LangGraph, validation, LlamaIndex, CrewAI	Proceed one gate at a time; do not install framework runtime packages in one combined step.
Blocked / gated	Live tracing, runtime installs, broad indexing, cloud API calls	These need provider credentials, approved source packets, or explicit approval before execution.

Verified Current State

- ArchFlow root is not a Git repository; public/ is the clean Git repository for the public-safe project.
- public/project/.env.local, public/project/.env.langsmith.local, and public/project/local/ are ignored by Git.
- Python 3.14.6 and uv 0.11.4 are available locally; future runtime should still pin the intended Python version with uv.
- Ollama is reachable when local socket access is allowed; Qwythos and gemma4:e4b are present in the local inventory.
- public/project/workflows/langgraph-controller.yaml, crewai-crew.yaml, and llamaindex-rag.yaml parse successfully.
- No pyproject.toml, uv.lock, requirements file, or package.json exists under public/project, so runtime packages should not be treated as installed code yet.
- LangSmith is configured as observability intent only; it still requires manual key insertion and public-safe source discipline before traces are sent.

Method Used

The plan combines the requested standup-style reporting pattern with strategic product planning and product-strategy session logic. Known facts, assumptions, decisions, risks, and validation steps are separated. The decomposition uses explicit phases, decision gates, and acceptance criteria. Notion receives operational task structure; this PDF gives the fuller report.

E1 Whole-Epic Plan

Task	Goal	Output	Primary check
E1.1	Set up clean KB project from scratch.	Working setup map, provider/config rules, gated runtime plan.	Repo/env/YAML/model readiness checks.
E1.2	Run one dialogue-to-summary-to-PRD proof.	Source inventory, context digest, PRD, task matrix, KB update packet, review report.	AF Review approves or returns packet for revision.

Task	Goal	Output	Primary check
E1.3	Write PRD and agent history into the KB.	Populated KB with source trace and agent readback result.	Agent can read back objective, current status, blockers, and next action.

E1.1 - Setup A Separate KB Project

E1.1 establishes the clean operating base. The mistake to avoid is installing LangGraph, CrewAI, LlamaIndex, tracing, validation, and provider integrations in one combined move. The task should progress in small checkpoints so each dependency is clear and reversible.

ID	Subtask	Output	Check / gate
E1.1.1	Git safety and current state snapshot	Baseline note on repo, ignored files, tool readiness.	Git clean check before runtime work.
E1.1.2	Env/config boundary and secret handling	Provider/config map with key names only.	Tracked examples only; no raw keys.
E1.1.3	Ollama local model readiness	Local provider routing rule.	Model inventory or smoke test.
E1.1.4	LangSmith tracing readiness	Public-safe tracing rule.	No live traces until key and source approval.
E1.1.5	LangGraph control scaffold plan	Nodes, state, interrupts, review gates.	YAML parses and every node has output/gate.
E1.1.6	Pydantic/YAML validation layer	Schema plan for source, PRD, task, KB, review outputs.	Validation exists before RAG/team execution.
E1.1.7	LlamaIndex corpus and retrieval scaffold	Allowed corpus, metadata, citation requirements.	No broad/private indexing.
E1.1.8	CrewAI team config after graph validation	AF agent role map and output contract.	No CrewAI memory/delegation until proof stable.
E1.1.9	Observability and provenance paper pack	Setup summary, checks, gaps, provenance rules.	Documentation does not overclaim runtime install.

E1.2 - Run The Dialogue To PRD Cycle

E1.2 is the proof run. The input should be one approved, sanitized source packet. The output should be a complete packet that can move to the KB without requiring the raw source to be reread.

ID	Subtask	Output	Check / gate
E1.2.1	Select and approve first source packet	Scoped source manifest.	No whole-vault or raw private export.
E1.2.2	AF Tools inventory and extraction plan	Source inventory and extraction method.	Every source has reason and sensitivity.
E1.2.3	AF Context summary and decision log	FACT/INTERPRETATION/HYPOTHESIS/GAP digest.	No hypothesis presented as fact.
E1.2.4	Draft PRD with acceptance criteria	First proof PRD.	Can guide work without raw source.
E1.2.5	Responsibility matrix and task list	Owners, dependencies, checks.	No ownerless tasks.
E1.2.6	Tool use, provenance, run notes	Repeatable run log without private raw data.	Commands/checks and skipped checks recorded.
E1.2.7	AF Review and revise/approve	Review report with approve/block/revise.	E1.3 waits for approval or revision scope.

E1.3 - Write PRD And Agent History Into The KB

E1.3 turns the proof into durable memory. The output must be findable, source-backed, and safe. The readback test is mandatory because a KB that cannot be used by an agent is just a document pile.

ID	Subtask	Output	Check / gate
E1.3.1	Write approved PRD into KB structure	Findable PRD entry.	Agent can identify source/date/status.

ID	Subtask	Output	Check / gate
E1.3.2	Record agent history and decision trail	Agent outputs and decisions.	No raw logs/transcript dump.
E1.3.3	Update source registry and provenance map	Source-status map for durable claims.	Unsupported claims blocked.
E1.3.4	Add retrieval/index metadata	Metadata for future RAG inclusion/exclusion.	No broad/private indexing.
E1.3.5	Run agent readback test	Readback result.	Specific answer with facts and gaps.
E1.3.6	Prepare memory and insight candidates	Reusable memory/insight packet.	Evidence and future-use value required.
E1.3.7	Handoff to E1.4 and E2	Next-workstream handoff.	E1.4/E2 can start without missing context.

Decisions

- E1.1 is the current execution focus and should be treated as In Progress.
- Runtime installs should be separated from planning contracts. Current YAML/config contracts are not proof that LangGraph, LlamaIndex, or CrewAI packages are installed.
- LangGraph should control the path before CrewAI runs role/team work.
- Validation must come before retrieval and agent-team execution.
- LangSmith must remain public-safe tracing only until credentials and sanitized source rules are approved.

Risks And Mitigations

Risk	Mitigation
Bulk install creates unclear failures.	Install or activate one layer only after the previous layer passes its check.
Private material leaks into public project or traces.	Use source manifests, sensitivity labels, ignored local env files, and AF Review gate.
CrewAI runs without control path.	Keep CrewAI inside LangGraph or manual Codex supervision until graph and validation are stable.
RAG indexes too broad a corpus.	Use explicit allowed corpus and require approval for private/raw/broad sources.
Documentation overclaims runtime readiness.	Separate planned, configured, installed, and verified states in every report.

Immediate Next Steps

- Complete E1.1.2 by finalizing public-safe env/config boundaries and provider status labels.
- Move E1.1.3 only after confirming whether Ollama will run a small smoke test for this proof or remain readiness-only.
- Keep LangSmith inactive until a sanitized trace packet is approved.
- Define the first Pydantic/YAML schemas before any LlamaIndex/CrewAI runtime execution.
- Use the new archflow-task-breakdown skill for future E1.4/E2 task decomposition.

Artifacts Updated

- Notion E1 epic page: added 2026-06-25 execution planning update.
- Notion E1.1, E1.2, E1.3 parent task pages: added detailed execution plans.
- Notion task database: created 23 subtasks across E1.1 through E1.3.
- Public skill: [public/skills/archflow-task-breakdown/SKILL.md](#).
- This PDF report: [public/project/reports/2026-06-25-e1-block1-subtask-execution-plan.pdf](#).